Evidence Pack: Automating Secure Onboarding at Scale

Onward Logistics, Week 1. Prepared by Effie Njoki, 19 October 2026.

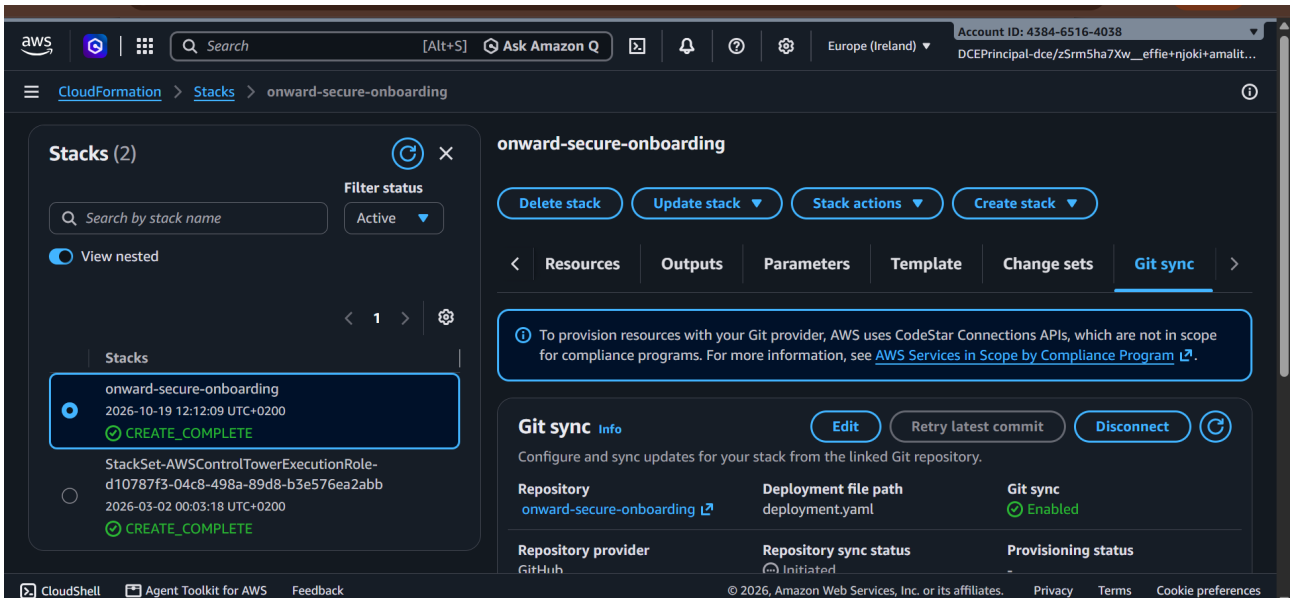
This pack shows that each client objective is met. No passwords, keys or secrets appear in any screenshot.

What the client checks	Evidence
Each engineer can do their job on day one, and cannot do more	04, 06, 09, 10
The on-call engineer still cannot create or terminate compute after extra permissions are added to their group or user	03, 05
Secure first login with no shared plain text credential and a forced change	07, 08
The Head of Security can see how access was defined and who changed it	11, 12, 13
Everything provisioned with CloudFormation via Git sync, and rebuildable from the repository	01, 02, 12

Note on names: these screenshots were taken before the company naming standard was applied. Groups and boundaries were then renamed from onward-... to onward-dev-... in a later commit (for example, onward-boundary-oncall became onward-dev-boundary-oncall). The permissions themselves did not change.

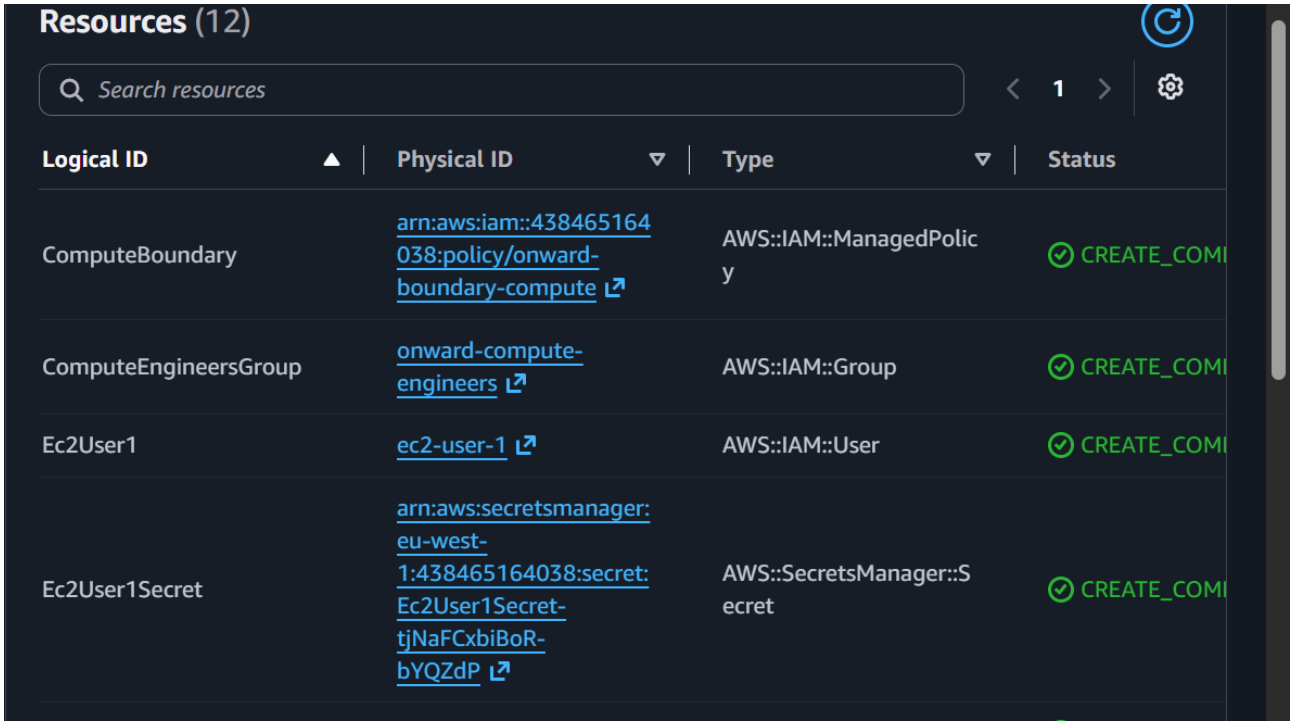
01. Stack created through Git sync

Proves: The stack onward-secure-onboarding reached CREATE_COMPLETE and Git sync is enabled for the GitHub repository. Access is deployed from version-controlled code, not created by hand.



02. All 12 resources built by CloudFormation

Proves: Secrets, boundaries, groups and users were all created by the stack (CREATE_COMPLETE). Nothing was created manually in the console.



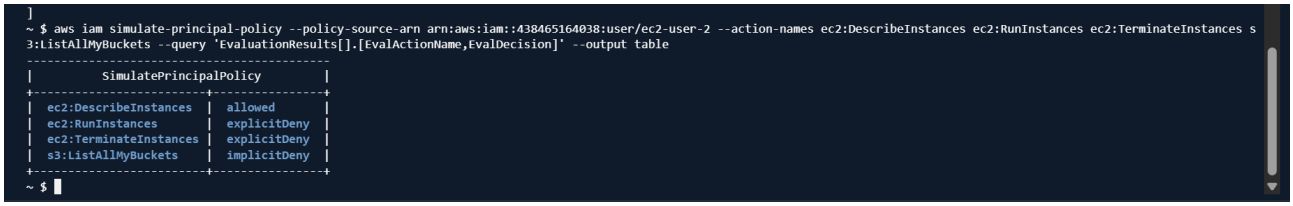
03. On-call engineer: boundary on the user, member of the on-call group

Proves: ec2-user-2 has the on-call permissions boundary attached directly to the user, and belongs to the on-call group. Because the cap sits on the user, changing the group cannot remove it.



04. On-call engineer can observe, never create or terminate

Proves: IAM policy simulator for ec2-user-2: DescribeInstances is allowed; RunInstances and TerminateInstances are explicitDeny (from the boundary); S3 is implicitDeny.



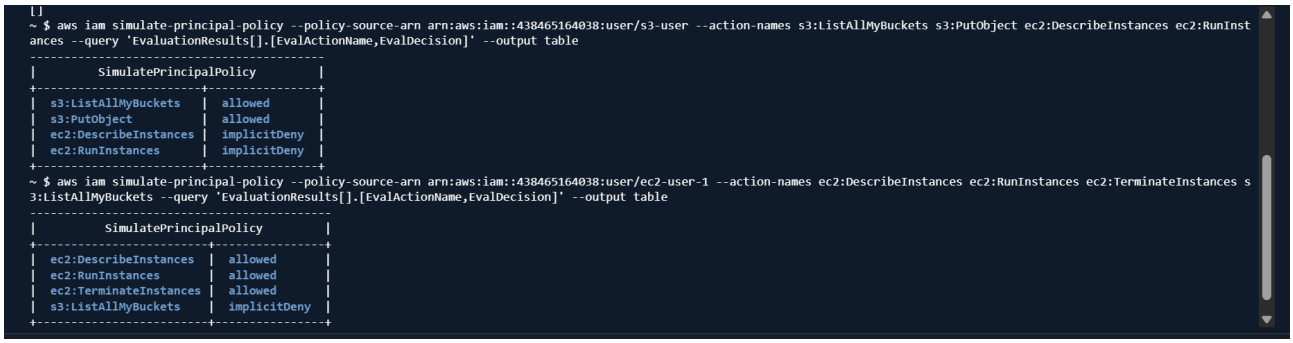
05. Key test: guarantee holds after permissions are loosened

Proves: AmazonEC2FullAccess was attached to BOTH the on-call group and the user as a test. RunInstances and TerminateInstances are still explicitDeny, because the boundary caps the user and an explicit Deny always wins. The test permissions were detached afterwards (the empty list at the top of evidence 06).



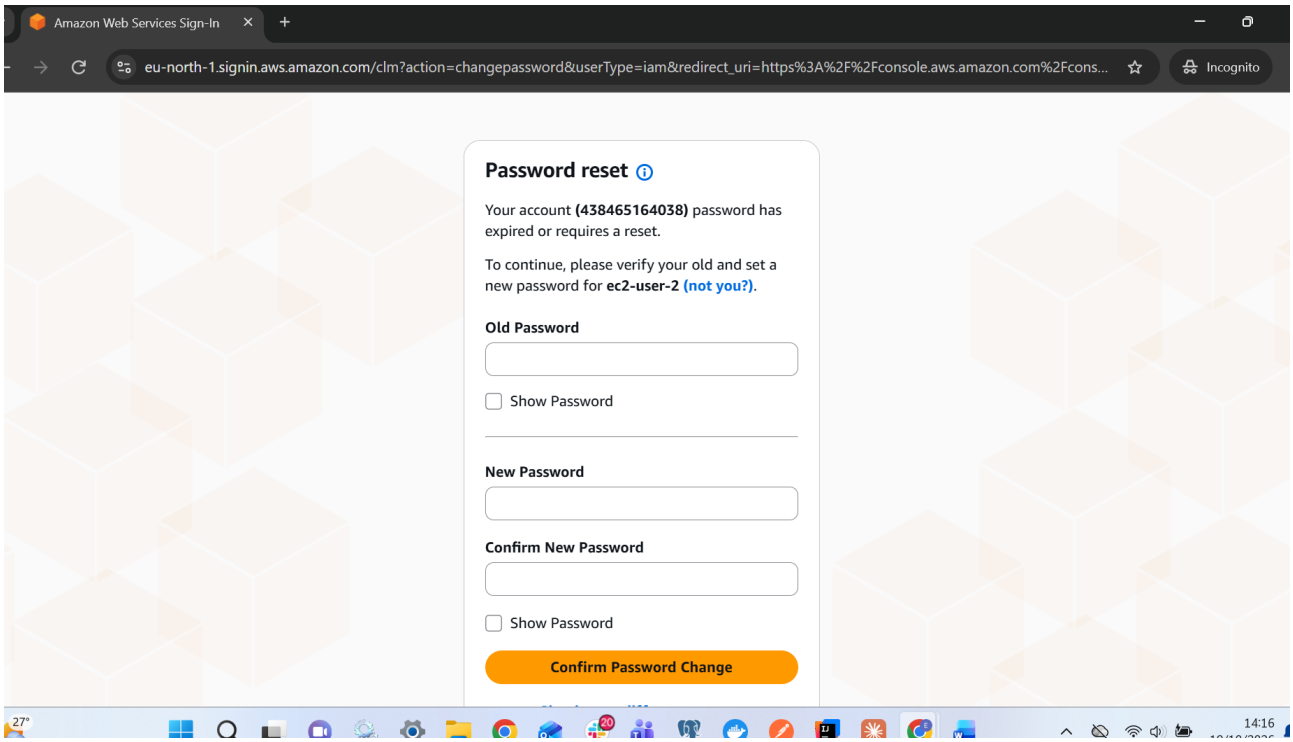
06. Each engineer can do their job, and no more

Proves: s3-user: S3 allowed, EC2 implicitDeny. ec2-user-1: EC2 allowed (including launch and terminate, as the compute engineer), S3 implicitDeny.



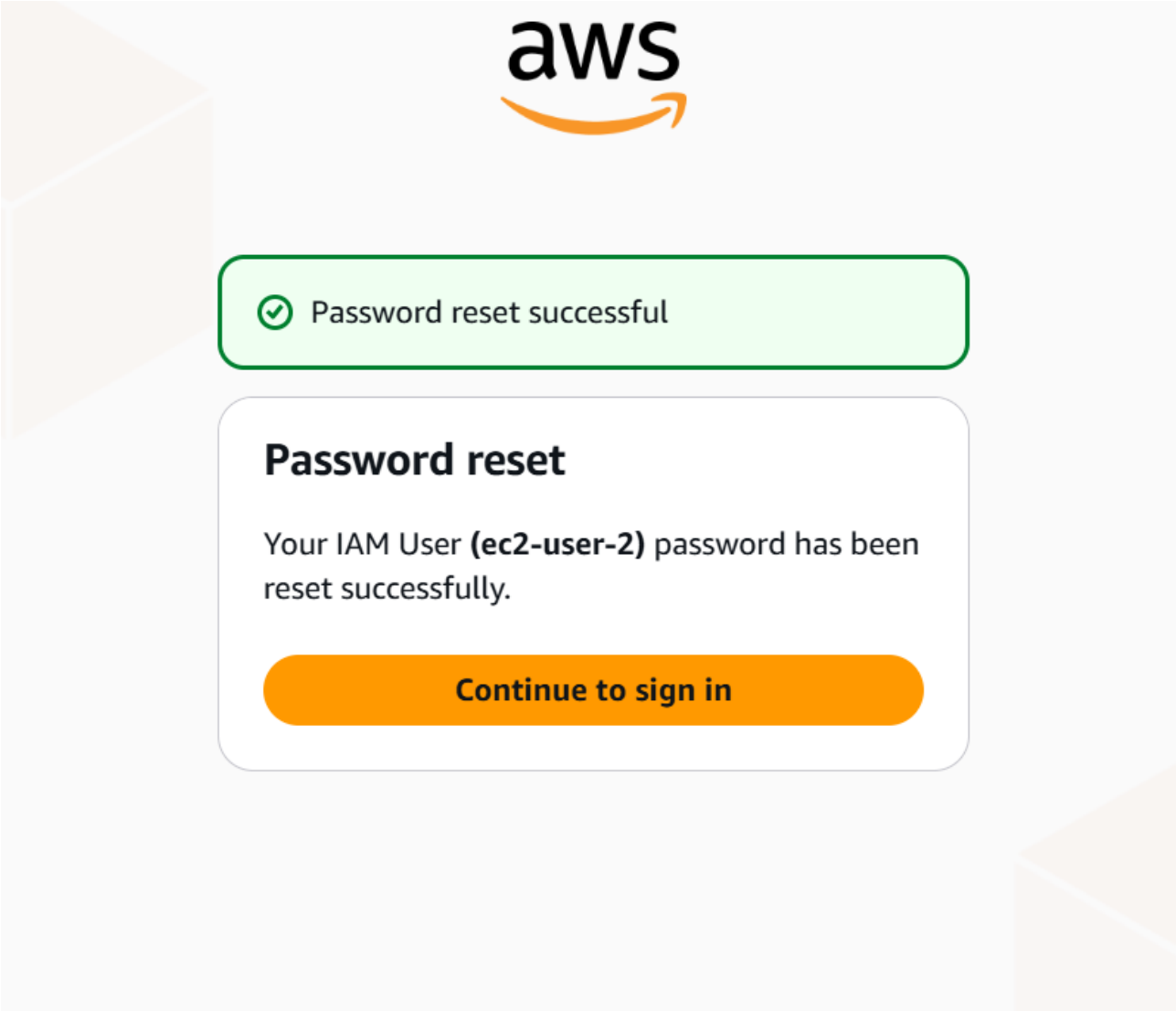
07. Secure first login: forced password change

Proves: Signing in as ec2-user-2 with the password generated by Secrets Manager immediately requires a new password. No shared or plain text credential is used.



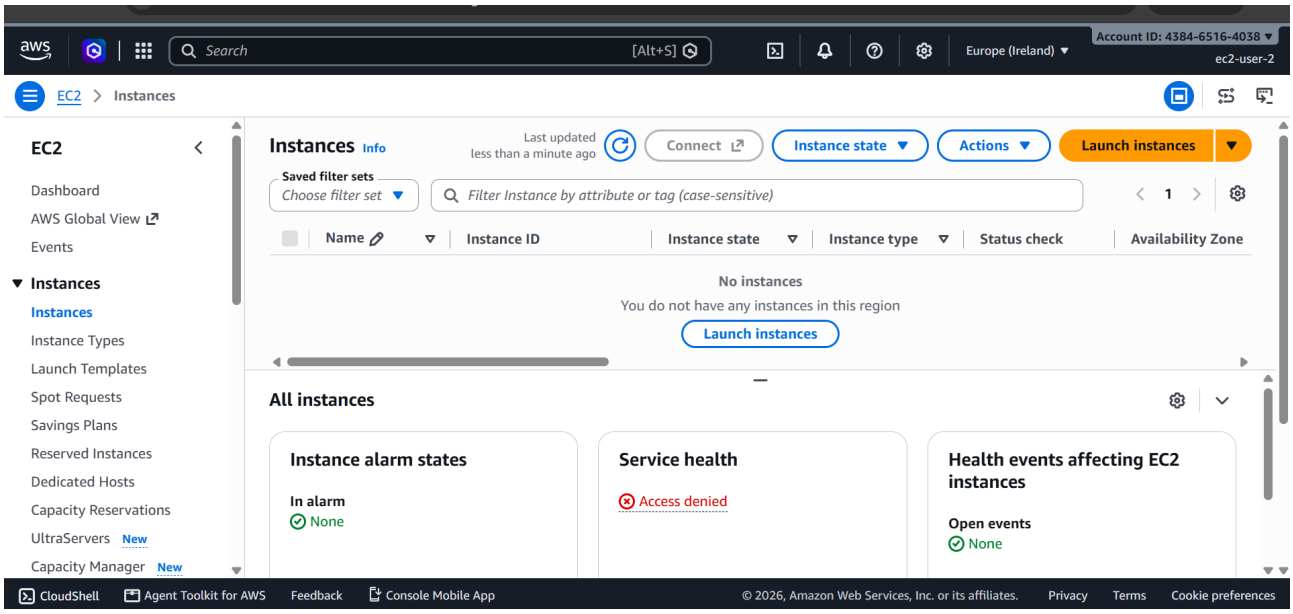
08. Password changed by the engineer

Proves: The boundary allows users to change only their own password, so the forced change succeeds. The original generated password no longer works.



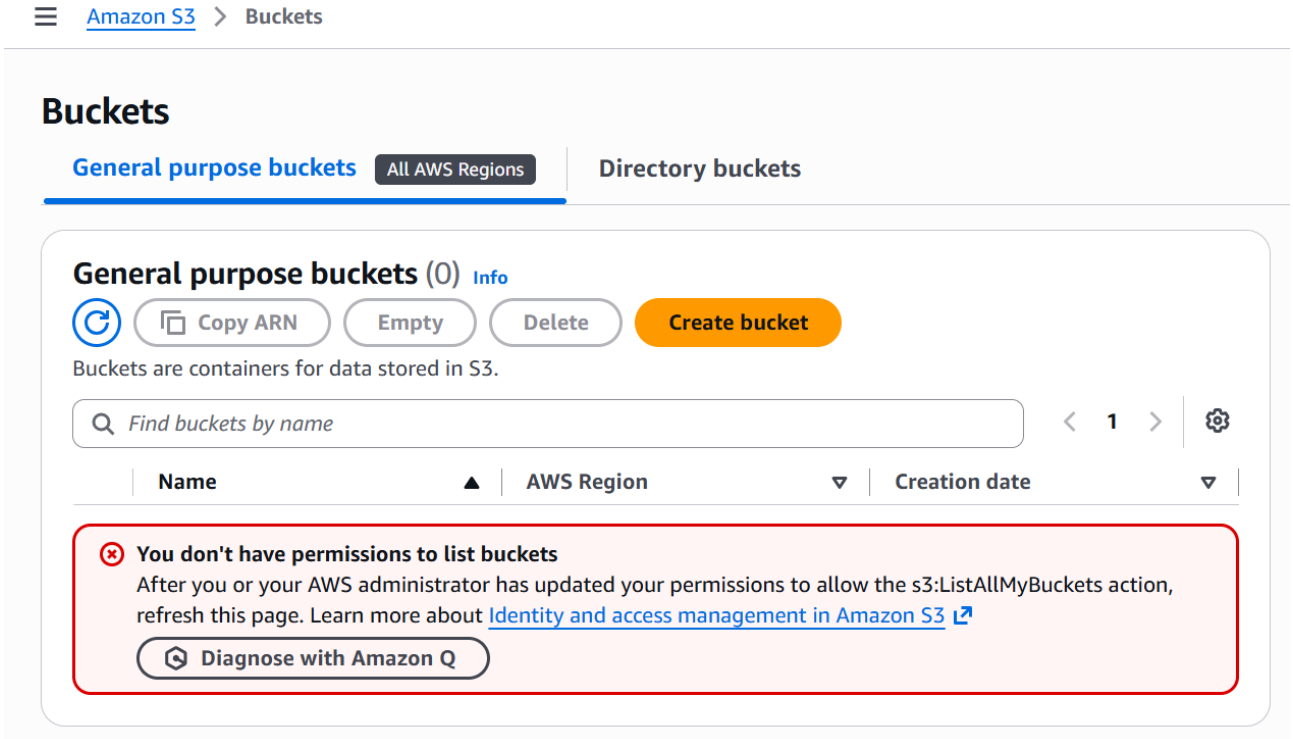
09. Real sign-in: on-call engineer can view EC2

Proves: Signed in as ec2-user-2 (top right), the EC2 Instances page loads. The AWS Health panel shows Access denied, an example of least privilege.



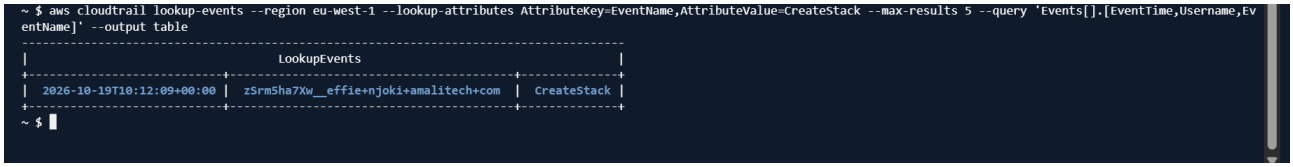
10. Real sign-in: on-call engineer cannot use S3

Proves: As ec2-user-2, listing buckets is denied (s3:ListAllMyBuckets), matching the simulator result.



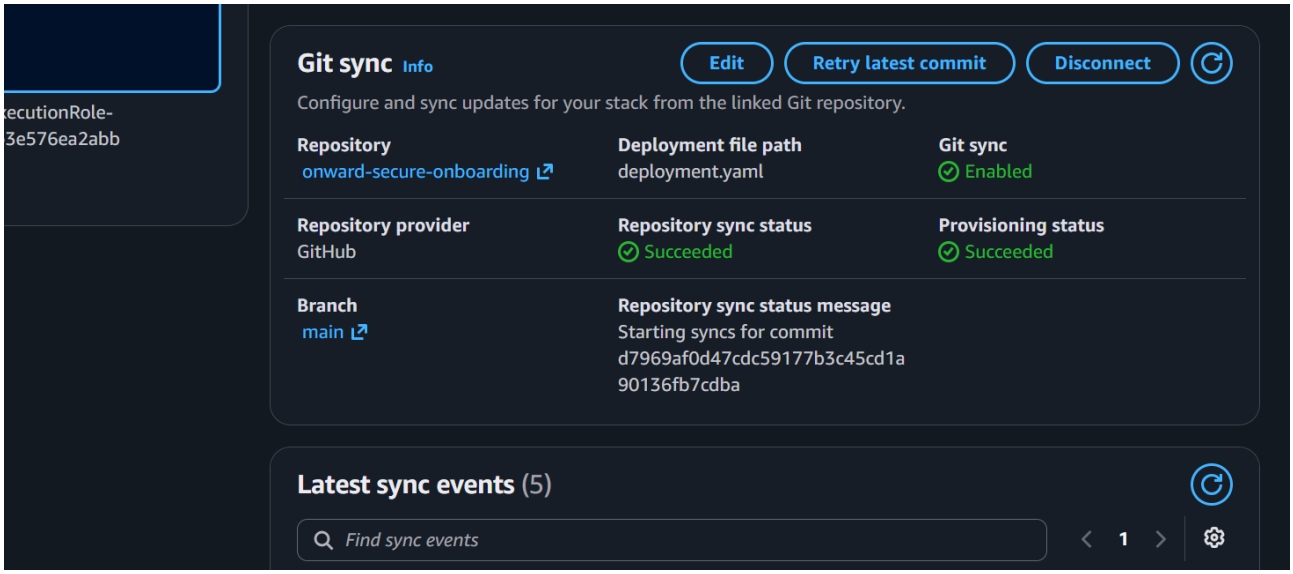
11. Audit trail: deployment traced to a person and a time

Proves: CloudTrail records the CreateStack call with the exact time and the deploying user session.



12. Audit trail: which Git commit was deployed

Proves: Git sync shows repository sync and provisioning Succeeded for commit d7969af, linking the deployment to a specific change in Git.



13. Audit trail: how access is defined and who changed it

Proves: The GitHub repository shows commit d7969af by effienjoki, the same commit deployed in evidence 12. Every change to access is a commit with an author and a time.

